Progress Report 1: Metroidvania-Style Procedural Map Generation

Course: CS5800 — Algorithms
Team Name: HACHIMI
Team Members: Monica Kumaran, Changfeng Wang, Tianyu Ma, Jinghao Zheng
Project Link: https://github.com/CS5800GroupHACHIMI/MapGeneration
Date: March 27, 2026

---

1. Executive Summary / Overview

The HACHIMI team is building a procedural 2D dungeon map generator inspired by Metroidvania games such as *Dead Cells* and *Hollow Knight*. The project is implemented in Unity (C#) and currently features a working map generation pipeline with three distinct generators: a Metroidvania-style room graph generator, a random room generator, and a single room generator. The core architecture—including the generator interface, tile system, and grid model—is fully operational, and the team is on track to enter the algorithm expansion and comparison phase. No major blockers have been encountered so far.

---

2. Problem Statement

Problem: Given a 2D grid of configurable dimensions, procedurally generate a dungeon-style map that is both random and structurally playable—meaning all rooms are reachable, the layout feels natural, and a player spawn point is well-defined.

Inputs:

- `MapConfig`: a configuration object specifying grid width, grid height, a random seed (or flag for random seed generation), and a default tile type.
- Algorithm selection: the user chooses which generator strategy to apply (e.g., Metroidvania, Random Room, Single Room).

Outputs:

- `MapGrid`: a 2D tile grid where each cell is assigned a `TileType` (Air, Wall, or Floor), representing the generated dungeon map.
- A `Vector2Int` start position indicating where the player spawns.
- A visual rendering of the map in Unity via Tilemap.

Constraints:

- The map must be fully connected (no isolated, unreachable rooms) in the Metroidvania generator.
- Room dimensions, door sizes, and room counts are bounded by constants to ensure playability.
- The generation must be deterministic for a given seed (reproducibility).
- The algorithm must run efficiently enough for real-time generation in Unity (target: sub-second for grids up to 64×64).

---

3. Algorithm Design

3.1 Approach

The project employs a **Strategy Pattern** with multiple map generation algorithms sharing a common interface (`IMapGenerator`). The primary algorithm—`MetroidvaniaGenerator`—uses a combination of **randomized graph expansion (random walk on a grid graph)** and **room carving** to produce Metroidvania-style maps. This approach can be classified as a **randomized greedy / graph-based procedural generation** algorithm.

The high-level strategy:

1. **Grid Partitioning:** The tile grid is conceptually divided into a grid of room cells, each of fixed size (10×8 tiles).
2. **Room Graph Construction (Random Walk):** Starting from the center cell, rooms are expanded outward via a randomized frontier-based walk, similar to a randomized BFS/Prim's expansion.
3. **Loop Injection:** Extra connections are added between existing adjacent rooms to create cycles, preventing a pure tree structure and improving navigability.
4. **Tile Carving:** Each room's interior is carved to Floor tiles, and doorways are opened between connected rooms.

3.2 Pseudocode

```
METROIDVANIA-GENERATE(grid, config):
    // Seed initialization (handled by MapGeneratorRunner before this call)
    IF config.randomSeed:
        config.seed ← Random integer in [INT_MIN, INT_MAX]
    Random.InitState(config.seed)
    cols ← max(2, grid.Width / RoomW)
    rows ← max(2, grid.Height / RoomH)
    Fill entire grid with Wall tiles

    // Phase 1: Build Room Graph
    startCol ← cols / 2
    startRow ← rows / 2
    targetRooms ← clamp(cols * rows / 2, 4, 30)

    hasRoom[startCol, startRow] ← true
    frontier ← { (startCol, startRow) }
    count ← 1

    WHILE count < targetRooms AND frontier is not empty:
        Pick random room (cur) from frontier
        Shuffle direction list [Right, Left, Up, Down]
        FOR EACH direction d:
            (nx, ny) ← neighbor of cur in direction d
            IF (nx, ny) is in bounds AND has no room:
                hasRoom[nx, ny] ← true
                Record connection between cur and (nx, ny)
                Add (nx, ny) to frontier
                count ← count + 1
                BREAK
```

```
        IF no expansion possible:
            Remove cur from frontier

    // Phase 2: Add Extra Connections (Loops)
    extras ← max(1, count / 3)
    REPEAT up to extras * 15 attempts:
        Pick random existing room and random direction
        IF neighbor exists AND no connection yet:
            Add connection
            extras ← extras - 1

    // Phase 3: Carve Rooms and Doors
    FOR EACH room cell (cx, cy) with hasRoom = true:
        Set interior tiles (excluding border) to Floor

    FOR EACH horizontal connection:
        Carve door opening through shared vertical wall

    FOR EACH vertical connection:
        Carve door opening through shared horizontal wall

    Set startPosition ← center of starting room
```

3.3 Time and Space Complexity

Let **W** = grid width, **H** = grid height, **C** = number of room columns (W/RoomW), **R** = number of room rows (H/RoomH), and **N** = C × R (total possible rooms).

Time Complexity:

Phase	Complexity	Explanation
Grid initialization (fill with Wall)	O(W × H)	Iterates over every tile
Room graph construction (random walk)	O(N)	Each room is visited at most once; constant-time direction checks
Loop injection	O(N)	At most O(N) extras with O(1) per attempt
Room carving	O(N × RoomW × RoomH) = O(W × H)	Each room's interior is carved
Door carving	O(N × DoorSize) = O(N)	Constant-size doors per connection
Total	**O(W × H)**	Dominated by the grid initialization and room carving passes

Space Complexity:

Structure	Size	Explanation
MapGrid (tiles + types)	O(W × H)	Two 2D arrays of size W × H

Structure	Size	Explanation
hasRoom	O(C × R) = O(N)	Boolean grid for room cells
connH, connV	O(N)	Connection arrays
Frontier list	O(N) worst case	At most N rooms in frontier
Total	**O(W × H)**	Dominated by the tile grid

---

4. Work Completed

4.1 Input Generation Close to Real Data

The system uses Unity's `MapConfig` ScriptableObject to define realistic map parameters. Testing has been performed with the configurations below, which are representative of actual Metroidvania game room layouts. The random seed system allows both deterministic (fixed seed) and non-deterministic (random seed) generation, simulating real-world use cases where designers may want reproducible or varied layouts.

Config	Width	Height	Seed	Generator
Baseline	32	32	0	SingleRoom
Metroidvania (small)	32	32	42	Metroidvania
Metroidvania (target)	64	64	42	Metroidvania
Random layout	32	32	0	RandomRoom
Reproducibility check	64	64	42 (run ×2)	Metroidvania

4.2 Results and Outcomes

Three generators have been implemented and are producing valid output:

- **MetroidvaniaGenerator:** Produces a connected graph of rooms with doorways, cycles for navigability, and a well-defined player start position. On a 64×64 grid, the generator targets up to 24 rooms (computed as `clamp(cols × rows / 2, 4, 30)` where `cols = 6`, `rows = 8`), with multiple interconnections and loop-injection adding `max(1, count / 3)` extra connections (at least 1 guaranteed).
- **RandomRoomGenerator:** Produces a single bounded area with stochastic floor/wall placement (60% floor probability), useful as a baseline comparison.
- **SingleRoomGenerator:** Produces one large open room with walls on the border, serving as the simplest test case.

All generators correctly populate the `MapGrid` with appropriate tile types and define valid player start positions.

4.3 Implementation Progress

The following components are fully implemented:

- **Core Architecture:** `IMapGenerator` interface, `MapGeneratorBase` abstract class (Strategy Pattern)
- **Data Layer:** `TileType` enum (Air, Wall, Floor), `TileRegistry` ScriptableObject, `MapConfig` ScriptableObject
- **Model Layer:** `MapGrid` class with event-driven tile updates (`OnTileChanged`)
- **Generators:** `MetroidvaniaGenerator`, `SingleRoomGenerator` (both selectable via DI); `RandomRoomGenerator` (implemented and tested by temporarily swapping the registered type in `GameLifetimeScope`—not yet exposed in the Inspector enum)
- **Player Model:** `Player` (position model with `OnMoved` / `OnTeleported` events), `PlayerInput` (auto-generated Input System class), `PlayerController` (ITickable; reads input, validates bounds and non-Wall tiles, applies 0.12s move cooldown); `PlayerSpawner` exists as a class but is not registered in the DI container and is not called at runtime
- **Unity Integration:** Tilemap rendering, Cinemachine camera, VContainer dependency injection, DOTween animation
- **Tile Visual Upgrade:** Replaced simple `Tile` assets with Unity **Rule Tiles** (`Rule Floor.asset`, `Rule Wall.asset`) that automatically select the correct sprite variant based on neighboring tiles, producing context-aware visuals (walls face the correct direction, floors blend seamlessly). The "Desert Shooter Bundle (Sprites Only)" pixel art sprite sheet (16 PPU) was imported as the visual source.
- **Generator Selection Refactor:** Removed the `GeneratorType` enum and the `if/else` DI registration from `GameLifetimeScope.cs`. Each generator is now a Unity **ScriptableObject asset** (`Assets/Configs/Generators/*.asset`). The Inspector exposes a single `generator` field; any of the three assets can be dragged in to select the active generator without code changes. `RandomRoomGenerator` is now fully accessible at runtime (resolves the issue noted in Section 7).
- **Editor Tooling:** Added `PixelArtImportSettings.cs` (auto-applies 16 PPU / Point filter / RGBA32 uncompressed on texture import) and `PixelArtBatchReimport.cs` (menu item to force-reimport all pixel art textures) under `Assets/Editor/`.

(Full source code is attached in Section 10.)

4.4 Initial Testing

Basic test cases have been executed manually through Unity's play mode:

Test Case	Input	Expected Result	Status
Default Metroidvania generation	64×64, seed=0	Connected room graph with doors	✅ Pass
Seed reproducibility	64×64, seed=42 (run twice)	Identical maps both runs	✅ Pass
Random seed mode	64×64, randomSeed=true	Different map each run	✅ Pass

Test Case	Input	Expected Result	Status
Minimum grid size	32×32, Metroidvania	At least 4 rooms generated	✅ Pass
Single Room generation	32×32	One open room with wall border	✅ Pass
Random Room generation	32×32, seed=0	Mixed floor/wall interior with border	✅ Pass (tested via temporary DI swap; `RandomRoom` enum entry pending)
Player spawn validity	All generators	Player spawns on Floor tile	✅ Pass

5. Work in Progress

Task	Description	Responsible	% Complete
Additional algorithm research	Researching BFS, A*, Prim's, Kruskal's, and DFS for pathfinding and MST-based map solving	Monica, Tianyu	~30%
Graph representation	Implementing adjacency list representation of the room graph for pathfinding algorithms	Tianyu	~20%
Algorithm comparison framework	Designing metrics and test harness to compare different generation/solving algorithms	All	~10%

6. Planned Work / Next Steps

Task	Description	Responsible	Target Deadline
Implement pathfinding algorithms	Implement BFS, DFS, A* search on the room graph to find shortest/optimal paths between start and end points	Monica, Tianyu	Week 3 (Mar 30)
Implement MST algorithms	Implement Prim's and/or Kruskal's algorithm to find minimum spanning tree of the room graph	Changfeng, Monica	Week 3 (Mar 30)
Algorithm comparison & benchmarking	Compare all algorithms on metrics: path length, computation time, and map "quality" (connectivity, branching factor)	All	Week 4 (Apr 7)

Task	Description	Responsible	Target Deadline
Performance testing script	Automated testing across multiple seeds and grid sizes with timing data	All	Week 4 (Apr 7)
Final deliverables	Written report, presentation slides, code cleanup, and documentation	All	Week 5 (Apr 14)

---

7. Issues, Challenges & Risks

Problems Encountered

Issue	Impact	Resolution
`RandomRoomGenerator` not exposed in DI/Inspector	The `GeneratorType` enum in `GameLifetimeScope` only supported `SingleRoom` and `Metroidvania`; `RandomRoomGenerator` could not be selected at runtime without a code change	**Resolved (Mar 26).** The enum was removed entirely. `GameLifetimeScope` now exposes a `[SerializeField] MapGeneratorBase generator` field; each generator has a dedicated ScriptableObject asset in `Assets/Configs/Generators/`, and any of them can be dragged into the Inspector slot without modifying code.
Unity project structure complexity	The standard Unity folder hierarchy (Assets, Packages, ProjectSettings) adds overhead for team members less familiar with Unity	Jinghao created clear documentation and a well-defined interface (`IMapGenerator`) so other members can implement new algorithms without deep Unity knowledge

Potential Risks Going Forward

Risk	Likelihood	Mitigation
Algorithm integration complexity: pathfinding algorithms (A*, BFS) operate on a graph abstraction while the current system works on a tile grid	Medium	The room graph built in `MetroidvaniaGenerator` (hasRoom, connH, connV arrays) can be extracted into an adjacency list; this conversion layer needs to be implemented
Time pressure: 3 remaining weeks for 4 algorithms + comparison + final report	Medium	Algorithms are well-documented in CLRS; team members can work in parallel since the `IMapGenerator` interface allows independent development
Balancing randomness and playability in new generators	Low	The existing seed-based system and the loop-injection mechanism in MetroidvaniaGenerator provide a proven pattern to follow

8. Schedule Status

Status: On Track ✅

The team is currently at the end of Week 2 of the 5-week plan. The core Unity pipeline and initial map generation algorithms are complete, which aligns with the planned milestones for Weeks 1–2.

Week	Planned	Actual Status
Week 1 (Mar 10–16)	Research C# and Unity	✅ Completed
Week 2 (Mar 17–23)	Unity pipeline + random graph generation	✅ Completed (pipeline + 3 generators)
Week 3 (Mar 24–30)	Implement individual algorithms	🔄 In Progress
Week 4 (Mar 31–Apr 7)	Performance testing	Upcoming
Week 5 (Apr 8–14)	Written report + presentation	Upcoming

The team has slightly exceeded Week 2 expectations by delivering three working generators (Metroidvania, Random Room, Single Room) instead of just the random graph generation. This provides a solid foundation for the algorithm comparison work in Weeks 3–4.

9. Conclusion

The HACHIMI team has established a robust and extensible architecture for procedural dungeon map generation. The Strategy Pattern design allows each team member to independently develop and plug in new algorithms without modifying the core system. With three working generators, a functional Unity visualization pipeline, and a clear plan for the remaining three weeks, the team is confident in meeting the April 14 deadline. The primary focus for the next reporting period will be implementing pathfinding and minimum spanning tree algorithms on the room graph, followed by a comparative analysis of algorithm performance. No stakeholder decisions are needed at this time.

10. Attachments

10.1 Core Source Code

IMapGenerator.cs (Generator Interface)

```
using Model;
using UnityEngine;
namespace Generators
{
    public interface IMapGenerator
    {
        string Name { get; }
```

```
        void Generate(MapGrid grid, MapConfig config);
        Vector2Int GetStartPosition(MapGrid grid) =>
            new Vector2Int(grid.Width / 2, grid.Height / 2);
    }
}
```

MapGeneratorBase.cs (Abstract Base Class)

```
using Model;
using UnityEngine;

namespace Generators
{
    /// <summary>
    /// Inherit from this class to implement map generation algorithms.
    ///
    /// What we can use:
    ///   grid   – Use grid.Set(x, y, TileType.X) to place tiles
    ///   config – Map size, seed, default tile type
    ///
    /// What we need to do:
    ///   Set Name.
    ///   Implement Generate().
    ///   Assign _startPosition for the player spawn point.
    ///
    /// To add a new generator:
    ///   1. Create a class that extends MapGeneratorBase
    ///   2. Add [CreateAssetMenu] attribute
    ///   3. Create the asset in the Editor and assign it in GameLifetimeScope
    ///   – No other files need to change.
    /// </summary>
    public abstract class MapGeneratorBase : ScriptableObject, IMapGenerator
    {
        public abstract string Name { get; }

        protected Vector2Int _startPosition;

        public abstract void Generate(MapGrid grid, MapConfig config);

        public virtual Vector2Int GetStartPosition(MapGrid grid) => _startPosition;
    }
}
```

MetroidvaniaGenerator.cs (Primary Algorithm)

```
using Data;
using Model;
using System.Collections.Generic;
using UnityEngine;

namespace Generators
{
```

```
    /// <summary>
    /// Generates a Metroidvania-style room-based map.
    ///
    /// Algorithm:
    ///   1. Divide the map into a grid of room cells (RoomW x RoomH tiles each)
    ///   2. Starting from the center cell, expand via random walk to reach target room
count
    ///   3. Add extra connections (loops) so the layout isn't a pure tree
    ///   4. Draw each room (wall border + floor interior) and open doorways between
connected rooms
    ///
    /// Recommended MapConfig size: 64x64 or larger.
    /// </summary>
    [CreateAssetMenu(fileName = "MetroidvaniaGenerator", menuName =
"Generators/Metroidvania")]
    public class MetroidvaniaGenerator : MapGeneratorBase
    {
        public override string Name => "Metroidvania";

        private const int RoomW    = 10; // room width  in tiles (including 1-tile walls on
each side)
        private const int RoomH    = 8;  // room height in tiles (including 1-tile walls on
each side)
        private const int DoorSize = 2;  // door opening width/height in tiles

        // right, left, up, down
        private static readonly int[] Dx = {  1, -1,  0,  0 };
        private static readonly int[] Dy = {  0,  0,  1, -1 };

        public override void Generate(MapGrid grid, MapConfig config)
        {
            Random.InitState(config.seed);

            int cols = Mathf.Max(2, grid.Width  / RoomW);
            int rows = Mathf.Max(2, grid.Height / RoomH);

            var hasRoom = new bool[cols, rows];
            var connH   = new bool[cols - 1, rows];  // connH[cx,cy]: room(cx,cy) <->
room(cx+1,cy)
            var connV   = new bool[cols, rows - 1];  // connV[cx,cy]: room(cx,cy) <->
room(cx,cy+1)

            // Fill everything with solid Wall — rooms and corridors will be carved out
            for (int x = 0; x < grid.Width;  x++)
            for (int y = 0; y < grid.Height; y++)
                grid.Set(x, y, TileType.Wall);

            int startCol    = cols / 2;
            int startRow    = rows / 2;
            int targetRooms = Mathf.Clamp(cols * rows / 2, 4, 30);

            BuildRoomGraph(hasRoom, connH, connV, cols, rows, startCol, startRow,
targetRooms);
```

```
            // Carve room interiors (borders stay Wall = solid rock)
            for (int cx = 0; cx < cols; cx++)
            for (int cy = 0; cy < rows; cy++)
                if (hasRoom[cx, cy])
                    CarveRoom(grid, cx, cy);

            // Carve door openings through the shared walls between adjacent rooms
            for (int cx = 0; cx < cols - 1; cx++)
            for (int cy = 0; cy < rows; cy++)
                if (connH[cx, cy])
                    CarveDoorH(grid, cx, cy);

            for (int cx = 0; cx < cols; cx++)
            for (int cy = 0; cy < rows - 1; cy++)
                if (connV[cx, cy])
                    CarveDoorV(grid, cx, cy);

            // Player starts at the interior center of the starting room
            _startPosition = new Vector2Int(
                startCol * RoomW + RoomW / 2,
                startRow * RoomH + RoomH / 2
            );
        }

        // ── Room Graph ──────────────────────────────────────────────────

        private void BuildRoomGraph(
            bool[,] hasRoom, bool[,] connH, bool[,] connV,
            int cols, int rows, int startCol, int startRow, int targetRooms)
        {
            hasRoom[startCol, startRow] = true;
            var frontier = new List<Vector2Int> { new Vector2Int(startCol, startRow) };
            int count = 1;

            // Random-walk expansion: pick a random frontier room, try to grow in a random
direction
            while (count < targetRooms && frontier.Count > 0)
            {
                int fi  = Random.Range(0, frontier.Count);
                var cur = frontier[fi];

                var dirs = new List<int> { 0, 1, 2, 3 };
                Shuffle(dirs);
                bool expanded = false;

                foreach (int d in dirs)
                {
                    int nx = cur.x + Dx[d];
                    int ny = cur.y + Dy[d];

                    if (nx < 0 || nx >= cols || ny < 0 || ny >= rows) continue;
                    if (hasRoom[nx, ny]) continue;
```

```
                hasRoom[nx, ny] = true;
                SetConn(connH, connV, cur.x, cur.y, nx, ny, d);
                frontier.Add(new Vector2Int(nx, ny));
                count++;
                expanded = true;
                break;
            }

            if (!expanded) frontier.RemoveAt(fi);
        }

        // Add extra connections (loops) so the map has shortcuts and cycles
        int extras = Mathf.Max(1, count / 3);
        for (int attempt = 0; attempt < extras * 15 && extras > 0; attempt++)
        {
            int cx = Random.Range(0, cols);
            int cy = Random.Range(0, rows);
            if (!hasRoom[cx, cy]) continue;

            int d  = Random.Range(0, 4);
            int nx = cx + Dx[d];
            int ny = cy + Dy[d];

            if (nx < 0 || nx >= cols || ny < 0 || ny >= rows) continue;
            if (!hasRoom[nx, ny]) continue;
            if (IsConn(connH, connV, cx, cy, d)) continue;

            SetConn(connH, connV, cx, cy, nx, ny, d);
            extras--;
        }
    }

    private static void SetConn(bool[,] connH, bool[,] connV,
        int x, int y, int nx, int ny, int dir)
    {
        switch (dir)
        {
            case 0: connH[x,  y ] = true; break; // right: slot (x,   y)
            case 1: connH[nx, y ] = true; break; // left:  slot (x-1, y)
            case 2: connV[x,  y ] = true; break; // up:    slot (x,   y)
            case 3: connV[x,  ny] = true; break; // down:  slot (x,   y-1)
        }
    }

    private static bool IsConn(bool[,] connH, bool[,] connV, int x, int y, int dir)
    {
        switch (dir)
        {
            case 0: return x     < connH.GetLength(0) && connH[x,     y];
            case 1: return x - 1 >= 0                 && connH[x - 1, y];
            case 2: return y     < connV.GetLength(1) && connV[x,     y];
            case 3: return y - 1 >= 0                 && connV[x,     y - 1];
```

```
                default: return false;
            }
        }

        // ─── Drawing ──────────────────────────────────────────────────────────

        // Carve the interior of a room to Floor; the border tiles remain Wall (solid rock)
        private static void CarveRoom(MapGrid grid, int cx, int cy)
        {
            int ox = cx * RoomW;
            int oy = cy * RoomH;

            for (int x = ox + 1; x < ox + RoomW - 1; x++)
            for (int y = oy + 1; y < oy + RoomH - 1; y++)
                grid.Set(x, y, TileType.Floor);
        }

        // Carve a horizontal door: punch through the shared wall between room(cx,cy) and
room(cx+1,cy)
        private static void CarveDoorH(MapGrid grid, int cx, int cy)
        {
            int wallX  = cx * RoomW + RoomW - 1;  // right border of room(cx,cy)
            int midY   = cy * RoomH + RoomH / 2;
            int yStart = midY - DoorSize / 2;
            int yMin   = cy * RoomH + 1;
            int yMax   = cy * RoomH + RoomH - 2;

            for (int i = 0; i < DoorSize; i++)
            {
                int y = yStart + i;
                if (y < yMin || y > yMax) continue;
                grid.Set(wallX,     y, TileType.Floor);  // right wall of left room
                grid.Set(wallX + 1, y, TileType.Floor);  // left  wall of right room
            }
        }

        // Carve a vertical door: punch through the shared wall between room(cx,cy) and
room(cx,cy+1)
        private static void CarveDoorV(MapGrid grid, int cx, int cy)
        {
            int wallY  = cy * RoomH + RoomH - 1;  // top border of room(cx,cy)
            int midX   = cx * RoomW + RoomW / 2;
            int xStart = midX - DoorSize / 2;
            int xMin   = cx * RoomW + 1;
            int xMax   = cx * RoomW + RoomW - 2;

            for (int i = 0; i < DoorSize; i++)
            {
                int x = xStart + i;
                if (x < xMin || x > xMax) continue;
                grid.Set(x, wallY,     TileType.Floor);  // top    wall of bottom room
                grid.Set(x, wallY + 1, TileType.Floor);  // bottom wall of top    room
            }
```

```
        }

        private static void Shuffle<T>(List<T> list)
        {
            for (int i = list.Count - 1; i > 0; i--)
            {
                int j = Random.Range(0, i + 1);
                (list[i], list[j]) = (list[j], list[i]);
            }
        }
    }
}
```

RandomRoomGenerator.cs

```
using Data;
using Model;
using UnityEngine;

namespace Generators
{
    [CreateAssetMenu(fileName = "RandomRoomGenerator", menuName = "Generators/Random Room")]
    public class RandomRoomGenerator : MapGeneratorBase
    {
        public override string Name => "Random Room";

        // Probability that an interior cell becomes Floor (vs Wall)
        private const float FloorChance = 0.6f;

        public override void Generate(MapGrid grid, MapConfig config)
        {
            Random.InitState(config.seed);

            int margin = 1;

            for (int x = 0; x < grid.Width; x++)
            for (int y = 0; y < grid.Height; y++)
            {
                bool isBorder = x < margin || x >= grid.Width  - margin
                                           || y < margin || y >= grid.Height - margin;
                if (isBorder)
                {
                    grid.Set(x, y, TileType.Air);
                    continue;
                }

                bool isEdge = x == margin || x == grid.Width  - margin - 1
                                          || y == margin || y == grid.Height - margin - 1;
                if (isEdge)
                {
                    grid.Set(x, y, TileType.Wall);
                    continue;
```

```
                }

                grid.Set(x, y, Random.value < FloorChance ? TileType.Floor : TileType.Wall);
            }

            _startPosition = new Vector2Int(grid.Width / 2, grid.Height / 2);
        }
    }
}
```

SingleRoomGenerator.cs

```
using Data;
using Model;
using UnityEngine;

namespace Generators
{
    [CreateAssetMenu(fileName = "SingleRoomGenerator", menuName = "Generators/Single Room")]
    public class SingleRoomGenerator : MapGeneratorBase
    {
        public override string Name => "Single Room";

        public override void Generate(MapGrid grid, MapConfig config)
        {
            int margin = 2;

            for (int x = 0; x < grid.Width;  x++)
            for (int y = 0; y < grid.Height; y++)
            {
                bool isAir = x < margin || x >= grid.Width  - margin
                                        || y < margin || y >= grid.Height - margin;

                bool isWall = x == margin || x == grid.Width  - margin - 1
                                        || y == margin || y == grid.Height - margin - 1;
                if (isAir)
                    grid.Set(x, y, TileType.Air);
                else if (isWall)
                    grid.Set(x, y, TileType.Wall);
                else
                    grid.Set(x, y, TileType.Floor);
            }

            _startPosition = new Vector2Int(grid.Width / 2, grid.Height / 2);
        }
    }
}
```

MapConfig.cs (Configuration)

```
using Data;
using UnityEngine;
using UnityEngine.Serialization;
using UnityEngine.Tilemaps;

[CreateAssetMenu(fileName = "MapConfig", menuName = "Scriptable Objects/MapConfig")]
public class MapConfig : ScriptableObject
{
    public int width = 32;
    public int height = 32;

    public int seed = 0;
    public bool randomSeed = true;

    public TileType defaultMapTileData;
}
```

MapGrid.cs (Grid Model)

```
using System;
using Data;
using UnityEngine.Tilemaps;

namespace Model
{
    public class MapGrid
    {
        public int Width  { get; }
        public int Height { get; }

        private readonly TileBase[,]  _tiles;
        private readonly TileType[,]  _types;
        private readonly TileRegistry _registry;

        public event Action<int, int, TileBase> OnTileChanged;

        public MapGrid(MapConfig config, TileRegistry registry)
        {
            Width     = config.width;
            Height    = config.height;
            _tiles    = new TileBase[Width, Height];
            _types    = new TileType[Width, Height];
            _registry = registry;
        }

        public TileBase Get(int x, int y) => _tiles[x, y];

        public TileType GetTileType(int x, int y) => _types[x, y];

        public void Set(int x, int y, TileType type)
```

```
        {
            _types[x, y] = type;
            _tiles[x, y] = _registry.Get(type);
            OnTileChanged?.Invoke(x, y, _tiles[x, y]);
        }

        public void Reset(TileType defaultData)
        {
            for (int x = 0; x < Width;  x++)
            for (int y = 0; y < Height; y++)
                Set(x, y, defaultData);
        }

        public bool InBounds(int x, int y) =>
            x >= 0 && x < Width && y >= 0 && y < Height;
    }
}
```

TileType.cs (Tile Enumeration)

```
namespace Data
{
    public enum TileType
    {
        Air,
        Wall,
        Floor
    }
}
```

TileRegistry.cs (Tile Lookup)

```
using System;
using System.Collections.Generic;
using Data;
using UnityEngine;
using UnityEngine.Tilemaps;

[CreateAssetMenu(fileName = "TileRegistry", menuName = "Scriptable Objects/TileRegistry")]
public class TileRegistry : ScriptableObject
{
    [Serializable]
    private struct Entry
    {
        public TileType type;
        public TileBase data; // Tile / RuleTile / AnimatedTile
    }

    [SerializeField] private Entry[] entries;

    private Dictionary<TileType, TileBase> map;
```

```
    private void OnEnable()
    {
        map = new Dictionary<TileType, TileBase>();
        foreach (var e in entries)
            map[e.type] = e.data;
    }

    public TileBase Get(TileType type)
    {
        if (map.TryGetValue(type, out var data)) return data;
        Debug.LogError($"[TileRegistry] TileType.{type} not registered.");
        return null;
    }

    // For Special Tile
    public bool TryGetAs<T>(TileType type, out T tile) where T : TileBase
    {
        if (map.TryGetValue(type, out var data) && data is T cast)
        {
            tile = cast;
            return true;
        }
        tile = null;
        return false;
    }
}
```

TypedRuleTile.cs (Rule Tile with Neighbor-Type Awareness)

```
using System.Collections.Generic;
using Data;
using UnityEngine;
using UnityEngine.Tilemaps;

/// <summary>
/// A RuleTile that can match specific TileType neighbors (Air, Wall, Floor, …).
///
/// How neighbor constants work
/// ---------------------------
/// Unity's built-in constants occupy 1 (This) and 2 (NotThis).
/// Our constants start at TypeOffset (3) and are laid out as:
///
///   neighbor value = TypeOffset + (int)TileType
///
/// To add a new TileType: add ONE constant to the Neighbor class following the
/// same pattern. RuleMatch never needs to change.
/// </summary>
[CreateAssetMenu(fileName = "TypedRuleTile", menuName = "Tiles/Typed Rule Tile")]
public class TypedRuleTile : RuleTile<TypedRuleTile.Neighbor>
{
    private const int NeighborTypeOffset = 3;
```

```
    public class Neighbor : RuleTile.TilingRule.Neighbor
    {
        public const int Air   = 3; // NeighborTypeOffset + (int)TileType.Air   (0)
        public const int Wall  = 4; // NeighborTypeOffset + (int)TileType.Wall  (1)
        public const int Floor = 5; // NeighborTypeOffset + (int)TileType.Floor (2)
    }

    [Tooltip("The shared TileRegistry asset. Assign in the Inspector.")]
    public TileRegistry tileRegistry;

    // Reverse lookup: TileBase asset → TileType. Built lazily; reset on OnEnable.
    private Dictionary<TileBase, TileType> _reverseRegistry;

    private void OnEnable() => _reverseRegistry = null;

    public override bool RuleMatch(int neighbor, TileBase tile)
    {
        // Delegate built-in This / NotThis to the base class
        if (neighbor < NeighborTypeOffset || tileRegistry == null)
            return base.RuleMatch(neighbor, tile);

        var targetType = (TileType)(neighbor - NeighborTypeOffset);

        // A null TileBase on the Tilemap means the cell is empty → Air
        if (tile == null)
            return targetType == TileType.Air;

        return BuildReverseRegistry().TryGetValue(tile, out var actualType)
               && actualType == targetType;
    }

    private Dictionary<TileBase, TileType> BuildReverseRegistry()
    {
        if (_reverseRegistry != null) return _reverseRegistry;

        _reverseRegistry = new Dictionary<TileBase, TileType>();
        foreach (TileType type in System.Enum.GetValues(typeof(TileType)))
        {
            if (tileRegistry.TryGetAs<TileBase>(type, out var tileBase))
                _reverseRegistry[tileBase] = type;
        }

        return _reverseRegistry;
    }
}
```

GameLifetimeScope.cs (DI Root)

```
using Generators;
using Model;
using UnityEngine;
using UnityEngine.Tilemaps;
```

```
using VContainer;
using VContainer.Unity;

public class GameLifetimeScope : LifetimeScope
{
    [Header("Map")]
    [SerializeField] private MapConfig          mapConfig;
    [SerializeField] private TileRegistry       tileRegistry;
    [SerializeField] private Tilemap            tilemap;
    [SerializeField] private TilemapBoardView   boardView;
    [SerializeField] private MapGeneratorRunner runner;
    [SerializeField] private MapGeneratorBase   generator;

    [Header("Player")]
    [SerializeField] private PlayerView         playerView;

    protected override void Configure(IContainerBuilder builder)
    {
        builder.RegisterInstance(mapConfig);
        builder.RegisterInstance(tileRegistry);
        builder.RegisterInstance(tilemap);
        builder.RegisterInstance(new PlayerInput());

        builder.Register<MapGrid>(Lifetime.Singleton);
        builder.Register<Player>(Lifetime.Singleton);

        builder.RegisterInstance(generator).AsImplementedInterfaces();

        builder.RegisterComponent(boardView);
        builder.RegisterComponent(runner);
        builder.RegisterComponent(playerView);

        builder.RegisterEntryPoint<PlayerController>();
    }
}
```

10.2 GitHub Push History

The repository contains 9 commits on the `main` branch as of the reporting date (March 27, 2026). The commit history below documents the progression from initial setup through full pipeline integration:

Commit	Description
`a06be6c`	HelloWorld — initial Unity project creation
`db11cba`	helloworld — early project scaffolding
`18cfa58`	Clarify Unity Version
`b2e5420`	Basic Pipeline Dev — `IMapGenerator` interface, `MapGrid`, `TileRegistry`, core architecture
`0bfad22`	fix readme

Commit	Description
`f3bc235`	fix readme
`b8bb620`	Add the map random generation function and modify the movement of character — adds `MetroidvaniaGenerator`, `RandomRoomGenerator`, `SingleRoomGenerator`, VContainer DI wiring, Cinemachine, player system
`8b690b4`	import tile assets, improve generator config — imports "Desert Shooter Bundle" pixel art sprites, adds Rule Tiles (`Rule Floor`, `Rule Wall`), creates ScriptableObject config assets for each generator, refactors `GameLifetimeScope` to use Inspector-assignable generator field, adds pixel art editor tooling
`48cbce3`	Change to Rule Tiles — updates `TileRegistry.asset` to reference the new Rule Tile assets

(Insert screenshot of the GitHub commit history page here — navigate to `https://github.com/CS5800GroupHACHIMI/MapGeneration/commits/main` and attach a capture showing author, date, and message for each commit.)

10.3 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                        Unity Scene                           │
│  ┌────────────────┐  ┌──────────────┐  ┌──────────────────┐  │
│  │ TilemapBoard   │  │ Cinemachine  │  │ Player + Input   │  │
│  │ View           │  │ (Camera)     │  │ (Controller)     │  │
│  └────────┬───────┘  └──────────────┘  └────────┬─────────┘  │
│           │                                     │            │
│           │ OnTileChanged(x, y, TileBase)       │            │
│  ┌────────┴─────────────────────────────────────┴─────────┐  │
│  │                 MapGrid (Model)                        │  │
│  │  - TileBase[,] _tiles    - TileType[,] _types          │  │
│  │  - Set(x, y, TileType)   - Get(x, y) → TileBase        │  │
│  │  - GetTileType(x, y)     - InBounds(x, y)              │  │
│  │  - Reset(TileType)                                     │  │
│  └────────────────────────┬───────────────────────────────┘  │
│                           │ Generate(MapGrid, MapConfig)     │
│  ┌────────────────────────┴───────────────────────────────┐  │
│  │       MapGeneratorBase : ScriptableObject, IMapGen     │  │
│  │  ┌────────────────┐  ┌──────────────┐  ┌──────────────┐  │  │
│  │  │ Metroidvania   │  │ RandomRoom   │  │ SingleRoom   │  │  │
│  │  │ Generator      │  │ Generator    │  │ Generator    │  │  │
│  │  └────────────────┘  └──────────────┘  └──────────────┘  │  │
│  └────────────────────────────────────────────────────────┘  │
│                           │                                  │
│  ┌────────────────────────┴───────────────────────────────┐  │
│  │      Data Layer (MapConfig, TileRegistry)              │  │
│  │  TileRegistry.Get(TileType) → TileBase                 │  │
│  │  TileRegistry.TryGetAs<T>(TileType, out T)             │  │
│  └────────────────────────┬───────────────────────────────┘  │
│                           │ RuleMatch(neighbor, TileBase)    │
│  ┌────────────────────────┴───────────────────────────────┐  │
```

```
│  │   TypedRuleTile : RuleTile  (Tile Visual System)      │  │
│  │  - Neighbor: Air=3, Wall=4, Floor=5                   │  │
│  │  - BuildReverseRegistry() → Dict<TileBase, TileType>  │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │           VContainer (Dependency Injection)           │  │
│  │  GameLifetimeScope.Configure() wires all singletons   │  │
│  │  generator field → drag any MapGeneratorBase asset    │  │
│  └──────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────┘
```

10.4 Team Contribution Summary

Member	Role	Phase 1 Contribution
Jinghao Zheng	Lead Architect	Designed and implemented the entire Unity framework: `IMapGenerator` interface, `MapGeneratorBase`, `MapGrid`, `MapConfig`, `TileRegistry`, `TileType`, VContainer DI setup, Tilemap rendering, Cinemachine camera, player system
Changfeng Wang	Algorithm Developer	Implemented `MetroidvaniaGenerator` (room graph + random walk + loop injection + carving), `RandomRoomGenerator` (stochastic floor placement), and `SingleRoomGenerator` (baseline)
Monica Kumaran	Researcher	Conducted research on CLRS Ch. 22 (BFS, DFS), Ch. 23 (Prim's, Kruskal's), and Hart et al. 1968 (A*) for upcoming pathfinding and MST implementation
Tianyu Ma	Researcher	Researched procedural generation techniques and algorithm strategies for balancing randomness with playability; preparing for graph abstraction layer implementation